

MATH 231 – Linear Algebra I

The Computational Unit, Part 4 ■ The Gram–Schmidt Process

Kiely Hall 326 ■ Instructor: Kevin Bedoya

About this document. The vector unit ended one of its sections with a promise. It had just shown that against an *orthogonal* basis, coordinates cost nothing to find, and then asked whether such a basis can always be manufactured out of an ordinary one. It said yes, named the procedure, and stopped. This part is that procedure.

Part	Sections	Subject
1	§1–§4	operation counting, Big-O, floating point, error
2	§5–§10	k -nearest neighbours
3	§11–§15	k -means clustering
4	§16–§20	the Gram–Schmidt process

A common confusion (Whose §16 is this?). Section numbering runs continuously through the computational unit, so a bare “§18.2” below means §18.2 *of this unit*. References into the vector unit or the matrix unit always name them.

Assumed. This part expects the whole vector unit and the first part of the matrix unit. Specifically — from the vector unit: span, linear independence, basis and dimension (§17–§19); the dot product and its bilinearity (§11.8, §21.1); the norm, unit vectors and normalisation (§11.9); projection and the orthogonal piece $\mathbf{v} - \text{proj}_{\mathbf{u}}(\mathbf{v})$ (§12.4, §12.6); orthogonality, orthonormality, and why an orthogonal basis is worth having (§16, §21.6); $\mathbb{R}^n$ and n -dimensional notation (§24). From the matrix unit: the coordinate problem written as a linear system (§1), and the same system compressed to the matrix–vector product $\mathbf{V}\mathbf{x} = \mathbf{u}$ (§2.4). From this unit: Big-O (§2.7) and FLOP counting (§4.6).

What you should be able to do afterwards. State what an orthonormal set is and write it with the Kronecker delta; explain why coordinates against an orthonormal basis are dot products and no system need be solved; carry out Gram–Schmidt by hand in $\mathbb{R}^2$ and $\mathbb{R}^3$; write its pseudocode; say exactly what happens when the input vectors are dependent, in exact arithmetic and in floating point; and count the algorithm’s operations and give its complexity.

16. Reading coordinates without solving

16.1. What the basis left us with

Take n linearly independent vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ in $\mathbb{R}^n$. Being independent and n of them, they span $\mathbb{R}^n$: every vector in the space is reachable as a linear combination of them, and — this is the part that matters — reachable in exactly one way. So for any $\mathbf{u} \in \mathbb{R}^n$ there are unique numbers $\alpha_1, \dots, \alpha_n$ with

$$\mathbf{u} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_n \mathbf{v}_n. \quad (1)$$

Those α ’s are the *coordinates* of $\mathbf{u}$ against that basis. The basis guarantees they exist. It does not tell you what they are.

From the vector unit. The matrix unit took up exactly this question. Writing (1) out one component at a time produces n simultaneous linear equations in the n unknowns $\alpha_1, \dots, \alpha_n$ (matrix unit §1), and stacking the $\mathbf{v}_j$ as the columns of a matrix compresses all n equations into the single statement $\mathbf{V}\mathbf{x} = \mathbf{u}$ (matrix unit §2.4). What neither section did was *solve* it.

That is the state we are in. We can pose the problem and we can write it compactly, and for $n = 2$ we can grind it out by hand. But nothing about the compact notation makes the work smaller. There are still n unknowns constrained by n equations that must all hold at once, and no equation can be settled in isolation — change one α and every component of the sum moves. For $n = 2$ that is a nuisance. For $n = 500$, which is a small number of features by the standards of §5.1, it is a serious computation.

So the honest summary is: a basis promises the coordinates are there, and then hands you a system of equations.

Unless we choose the basis more carefully.

16.2. Orthonormal vectors, and the Kronecker delta

From the vector unit. A set of vectors is **orthonormal** when the vectors are mutually orthogonal and each has length one (vector unit §16). For two of them, $\mathbf{q}_i$ and $\mathbf{q}_j$, that is two statements:

$$\langle \mathbf{q}_i, \mathbf{q}_j \rangle = 0 \quad (i \neq j), \quad \langle \mathbf{q}_i, \mathbf{q}_i \rangle = 1.$$

The first is orthogonality; the second is unit length, since $\langle \mathbf{q}, \mathbf{q} \rangle = \|\mathbf{q}\|^2$ and $\|\mathbf{q}\| = 1$.

Those two conditions get written together often enough that they have a shorthand.

Definition 16.1 (Kronecker delta). The **Kronecker delta** δ_{ij} is the function

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

With it, the whole of orthonormality is one equation:

$$\langle \mathbf{q}_i, \mathbf{q}_j \rangle = \delta_{ij}. \tag{2}$$

The delta is not deep — it is a lookup table with two entries. What makes it worth a name is what it does inside a sum. Every term of $\sum_j \beta_j \delta_{ji}$ is zero except the one where $j = i$, which survives with $\delta_{ii} = 1$:

$$\sum_{j=1}^n \beta_j \delta_{ji} = \beta_i. \tag{3}$$

A sum of n terms collapses to one. Read δ as an instruction: *keep the matching index, delete everything else*.

A common confusion (Two different δ 's in this unit). Part 2 used δ_i for the distance from the query point to $\mathbf{x}_i$ (§7.4, §8.3). That is an unrelated object that happens to wear the same letter. Tell them apart by the indices: the Kronecker delta always carries *two*, and it is only ever 0 or 1.

16.3. The coordinates fall out

Now suppose the n vectors we were handed are not merely independent but orthonormal. Call them $\mathbf{q}_1, \dots, \mathbf{q}_n$. They are still n independent vectors in $\mathbb{R}^n$ — orthogonal vectors are independent,

which is the vector unit's §21.2 — so they still span, and (1) still applies. Write it with new letters for the coordinates:

$$\mathbf{u} = \beta_1 \mathbf{q}_1 + \beta_2 \mathbf{q}_2 + \cdots + \beta_n \mathbf{q}_n. \quad (4)$$

Everything so far is identical to §16.1. Here is where it stops being identical: take the dot product of both sides with $\mathbf{q}_1$.

The dot product is bilinear (vector unit §21.1), so it distributes across the sum and pulls the scalars out:

$$\langle \mathbf{u}, \mathbf{q}_1 \rangle = \beta_1 \underbrace{\langle \mathbf{q}_1, \mathbf{q}_1 \rangle}_{=1} + \beta_2 \underbrace{\langle \mathbf{q}_2, \mathbf{q}_1 \rangle}_{=0} + \cdots + \beta_n \underbrace{\langle \mathbf{q}_n, \mathbf{q}_1 \rangle}_{=0} = \beta_1.$$

Every term but the first vanishes. Not because of anything about $\mathbf{u}$, and not because of any cleverness — they vanish because the basis was built so that they would.

Nothing distinguished $\mathbf{q}_1$, so dot with $\mathbf{q}_i$ instead and the same collapse happens around the i -th term. In the delta notation it is one line, using (2) and then the sifting property (3):

$$\langle \mathbf{u}, \mathbf{q}_i \rangle = \left\langle \sum_{j=1}^n \beta_j \mathbf{q}_j, \mathbf{q}_i \right\rangle = \sum_{j=1}^n \beta_j \langle \mathbf{q}_j, \mathbf{q}_i \rangle = \sum_{j=1}^n \beta_j \delta_{ji} = \beta_i.$$

Theorem 16.1 (Coordinates against an orthonormal basis). *Let $\{\mathbf{q}_1, \dots, \mathbf{q}_n\}$ be an orthonormal basis of $\mathbb{R}^n$ and let $\mathbf{u} \in \mathbb{R}^n$. Then the coordinates of $\mathbf{u}$ are its dot products with the basis vectors:*

$$\beta_i = \langle \mathbf{u}, \mathbf{q}_i \rangle, \quad \text{so} \quad \mathbf{u} = \langle \mathbf{u}, \mathbf{q}_1 \rangle \mathbf{q}_1 + \langle \mathbf{u}, \mathbf{q}_2 \rangle \mathbf{q}_2 + \cdots + \langle \mathbf{u}, \mathbf{q}_n \rangle \mathbf{q}_n.$$

No system was solved. The coordinates were *read off*.

16.4. What that saved

It is worth being precise about the size of the win, because it is not a small constant factor.

Against a general basis, the n coordinates are entangled: they are the solution of a coupled system, and you cannot know any one of them until you have essentially done the work for all of them. Against an orthonormal basis they are *independent of each other*. Coordinate β_7 is one dot product. You can compute it without ever looking at β_3 , compute the coordinates in any order, compute only the ones you want, or compute them on separate machines at the same time.

From the vector unit. The vector unit reached this conclusion for an orthogonal basis in §21.6, where the coordinate came out as

$$\alpha = \frac{\langle \mathbf{u}, \mathbf{w} \rangle}{\|\mathbf{u}\|^2}.$$

Orthonormality is that formula with the denominator deleted. Normalising each basis vector makes $\|\mathbf{q}_i\|^2 = 1$, and the division disappears — so an orthonormal basis is an orthogonal basis with the last bit of arithmetic already paid for. That section also observed that $\mathbf{u}$ is then the sum of its own shadows on the basis directions; Theorem 16.1 is that statement with unit-length shadows, since for $\|\mathbf{q}\| = 1$ the projection $\text{proj}_{\mathbf{q}}(\mathbf{u})$ is exactly $\langle \mathbf{u}, \mathbf{q} \rangle \mathbf{q}$.

Remark. Any basis will answer the question. A general basis $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ determines $\mathbf{u}$ just as completely as an orthonormal one, and (1) is not wrong. The two differ in what it costs to *use* them. If the goal is to stitch together a set of directions that walk you to an arbitrary $\mathbf{u}$, and to do it repeatedly, an orthonormal basis is the one to want: same information, no system.

This is only useful if we can get such a basis. Bases do not arrive orthonormal. Where would one even start?

17. Building the basis: one move

17.1. The move, recalled from $\mathbb{R}^2$

We already have the whole idea. It was built in the vector unit for two dimensions and never called an algorithm.

From the vector unit. Given $\mathbf{u} \neq \mathbf{0}$ and any $\mathbf{v}$, the projection $\text{proj}_{\mathbf{u}}(\mathbf{v})$ is the part of $\mathbf{v}$ that points along $\mathbf{u}$, taken as a vector (§12.4; the subscript names the vector being projected *onto*). Subtracting it leaves

$$\mathbf{w} = \mathbf{v} - \text{proj}_{\mathbf{u}}(\mathbf{v}), \quad (5)$$

and §12.6 showed that $\mathbf{w} \perp \mathbf{u}$. The reason is geometric rather than computational: $\text{proj}_{\mathbf{u}}(\mathbf{v})$ is precisely the amount of $\mathbf{v}$ that lies along $\mathbf{u}$, so once it is removed, what is left has no component in the $\mathbf{u}$ direction at all. Nothing is left to be orthogonal about.

Confirm it in one line. Using bilinearity and $\text{proj}_{\mathbf{u}}(\mathbf{v}) = (\langle \mathbf{u}, \mathbf{v} \rangle / \|\mathbf{u}\|^2) \mathbf{u}$,

$$\langle \mathbf{w}, \mathbf{u} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle - \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \langle \mathbf{u}, \mathbf{u} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle - \langle \mathbf{u}, \mathbf{v} \rangle = 0.$$

The subtraction is engineered to make this cancellation happen; the coefficient on $\mathbf{u}$ is chosen for no other purpose.

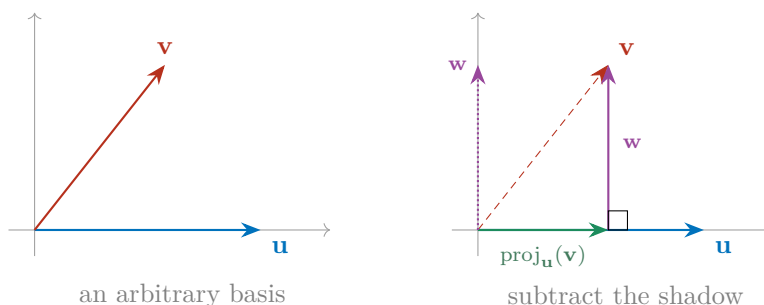
17.2. Two dimensions, finished

In $\mathbb{R}^2$ that single move has already done most of the job. Start with any basis $\{\mathbf{u}, \mathbf{v}\}$ — independent, otherwise arbitrary. Set $\mathbf{w} = \mathbf{v} - \text{proj}_{\mathbf{u}}(\mathbf{v})$. Then $\mathbf{u} \perp \mathbf{w}$, and we have two orthogonal vectors where we had two skew ones.

That is an orthogonal basis, not yet an orthonormal one: the lengths are still whatever they happened to be. Fix that by normalising each, which the vector unit's §11.9 does by dividing a vector by its own length:

$$\hat{\mathbf{q}}_1 = \frac{\mathbf{u}}{\|\mathbf{u}\|}, \quad \hat{\mathbf{q}}_2 = \frac{\mathbf{w}}{\|\mathbf{w}\|}.$$

Both now have length one, and scaling a vector does not change its direction, so they are still orthogonal. Two orthonormal vectors in $\mathbb{R}^2$, built out of an arbitrary basis, by one subtraction and two divisions.



The right-hand picture is the entire algorithm in miniature. Everything that follows is this move, applied again.

17.3. Three dimensions

Now take three independent vectors $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ in $\mathbb{R}^3$ and do it again, one vector at a time. The rule is the same at every step: *strip off everything that points along the directions already fixed*.

Step 1. There is nothing to be orthogonal to yet, so the first direction is free. Keep $\mathbf{v}_1$ and give it unit length:

$$\mathbf{w}_1 = \mathbf{v}_1, \quad \mathbf{q}_1 = \frac{\mathbf{w}_1}{\|\mathbf{w}_1\|}.$$

Step 2. Take $\mathbf{v}_2$ and remove its $\mathbf{q}_1$ component. Because $\mathbf{q}_1$ is already a unit vector, its projection coefficient needs no denominator — $\text{proj}_{\mathbf{q}_1}(\mathbf{v}_2) = \langle \mathbf{v}_2, \mathbf{q}_1 \rangle \mathbf{q}_1$:

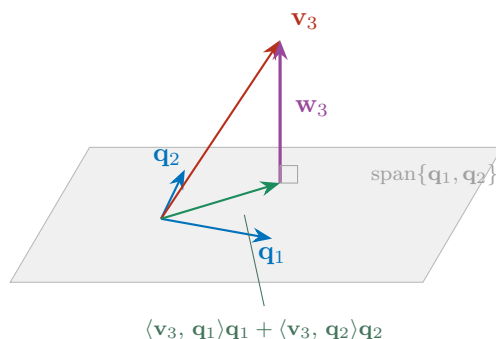
$$\mathbf{w}_2 = \mathbf{v}_2 - \langle \mathbf{v}_2, \mathbf{q}_1 \rangle \mathbf{q}_1, \quad \mathbf{q}_2 = \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}.$$

This is §17.2 verbatim. Now $\mathbf{q}_1 \perp \mathbf{q}_2$, both unit length, and together they span a *plane* through the origin — a two-dimensional subspace of $\mathbb{R}^3$, in the sense of the vector unit's §22.2.

Step 3. Here is the one genuinely new step, and the reason three dimensions is worth drawing. $\mathbf{v}_3$ sticks out of that plane. It has a component along $\mathbf{q}_1$ and a component along $\mathbf{q}_2$, and both must go:

$$\mathbf{w}_3 = \mathbf{v}_3 - \langle \mathbf{v}_3, \mathbf{q}_1 \rangle \mathbf{q}_1 - \langle \mathbf{v}_3, \mathbf{q}_2 \rangle \mathbf{q}_2, \quad \mathbf{q}_3 = \frac{\mathbf{w}_3}{\|\mathbf{w}_3\|}.$$

The two subtracted terms are together the *shadow of $\mathbf{v}_3$ on the plane*: they are exactly the coordinates Theorem 16.1 would read off if $\mathbf{v}_3$ lay in that plane. Removing them leaves the part of $\mathbf{v}_3$ that the plane cannot see — the vertical drop from the tip of $\mathbf{v}_3$ straight down to the plane.



Read the picture as a triangle: $\mathbf{v}_3$ is the hypotenuse from the origin, the green vector is its shadow lying flat in the plane, and $\mathbf{w}_3$ is the leg joining the shadow's tip to $\mathbf{v}_3$'s tip. That leg meets the plane at a right angle, so it is orthogonal to *everything* in the plane — to $\mathbf{q}_1$, to $\mathbf{q}_2$, and to every combination of them. Normalising it gives $\mathbf{q}_3$, and $\{\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3\}$ is an orthonormal basis of $\mathbb{R}^3$.

Remark. Notice what step 3 did *not* require: it did not go back and adjust $\mathbf{q}_1$ or $\mathbf{q}_2$. Once a direction is fixed it is never revisited. Each new vector is cleaned against the directions already built, and then it joins them. That is what makes the procedure a loop rather than a system.

18. The Gram–Schmidt process

The procedure just carried out in $\mathbb{R}^2$ and $\mathbb{R}^3$ is the **Gram–Schmidt process**. It takes a list of linearly independent vectors and returns an orthonormal list spanning the same space.

18.1. The general step

Suppose $\mathbf{v}_1, \dots, \mathbf{v}_n$ are linearly independent in $\mathbb{R}^n$, and suppose $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ have already been produced. To produce $\mathbf{q}_k$, subtract from $\mathbf{v}_k$ its component along every direction built so far, then normalise what is left:

$$\mathbf{w}_k = \mathbf{v}_k - \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{q}_j \rangle \mathbf{q}_j, \quad \mathbf{q}_k = \frac{\mathbf{w}_k}{\|\mathbf{w}_k\|}. \quad (6)$$

For $k = 1$ the sum is empty, so $\mathbf{w}_1 = \mathbf{v}_1$ and the first step is just a normalisation — consistent with §17.3, where the first direction was free.

That $\mathbf{w}_k$ really is orthogonal to each earlier $\mathbf{q}_i$ is the computation of §17.1 done $k - 1$ times at once. Dot (6) with $\mathbf{q}_i$ for some $i < k$, and use bilinearity together with $\langle \mathbf{q}_j, \mathbf{q}_i \rangle = \delta_{ji}$:

$$\langle \mathbf{w}_k, \mathbf{q}_i \rangle = \langle \mathbf{v}_k, \mathbf{q}_i \rangle - \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{q}_j \rangle \delta_{ji} = \langle \mathbf{v}_k, \mathbf{q}_i \rangle - \langle \mathbf{v}_k, \mathbf{q}_i \rangle = 0.$$

The delta collapses the sum to its $j = i$ term, which is precisely the quantity that was subtracted. Every earlier direction is cancelled, and only those.

Remark (The spans match at every stage). Gram–Schmidt does not merely produce *some* orthonormal basis — it produces one aligned with the input list, in the sense that

$$\text{span}\{\mathbf{q}_1, \dots, \mathbf{q}_k\} = \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} \quad \text{for every } k.$$

Each $\mathbf{q}_k$ is built only out of $\mathbf{v}_k$ and earlier $\mathbf{q}$'s, so it adds nothing new to the span; and $\mathbf{v}_k$ can be recovered from $\mathbf{q}_1, \dots, \mathbf{q}_k$ by undoing (6). The two lists sweep out the same nested sequence of subspaces — the same line, then the same plane, then the same three-dimensional space, and so on.

18.2. Pseudocode

Gram–Schmidt

Input: linearly independent $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathbb{R}^n$. **Output:** orthonormal $\mathbf{q}_1, \dots, \mathbf{q}_n$ with $\text{span}\{\mathbf{q}_1, \dots, \mathbf{q}_k\} = \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ for all k .

```

1: for  $k = 1, \dots, n$ :
2:    $\mathbf{w} \leftarrow \mathbf{v}_k$  start from the raw vector
3:   for  $j = 1, \dots, k - 1$ :
4:      $\mathbf{w} \leftarrow \mathbf{w} - \langle \mathbf{v}_k, \mathbf{q}_j \rangle \mathbf{q}_j$  strip the  $\mathbf{q}_j$  component
5:    $\mathbf{q}_k \leftarrow \mathbf{w} / \|\mathbf{w}\|$  normalise what survives
6: return  $\mathbf{q}_1, \dots, \mathbf{q}_n$ 

```

Two lines deserve a second look.

Line 4 dots against $\mathbf{v}_k$, the *original* vector, not against the partially-stripped $\mathbf{w}$. In exact arithmetic it makes no difference which you use: $\mathbf{w}$ differs from $\mathbf{v}_k$ only by multiples of $\mathbf{q}_1, \dots, \mathbf{q}_{j-1}$, all of which are orthogonal to $\mathbf{q}_j$ and so contribute nothing to the dot product. In floating-point arithmetic it makes a substantial difference. We return to this in §20.3.

Line 5 divides by $\|\mathbf{w}\|$, and divides without checking it. That is safe exactly when the inputs are independent, which is why the input line says so. §19 is about what happens when they are not.

18.3. A worked example

Example 18.1. Orthonormalise

$$\mathbf{v}_1 = [1, 1, 0], \quad \mathbf{v}_2 = [1, 0, 1], \quad \mathbf{v}_3 = [0, 1, 1].$$

$k = 1$. Nothing to subtract. $\|\mathbf{v}_1\| = \sqrt{2}$, so

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}}[1, 1, 0].$$

$k = 2$. $\langle \mathbf{v}_2, \mathbf{q}_1 \rangle = \frac{1}{\sqrt{2}}(1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0) = \frac{1}{\sqrt{2}}$, so

$$\mathbf{w}_2 = [1, 0, 1] - \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}}[1, 1, 0] = [1, 0, 1] - \frac{1}{2}[1, 1, 0] = \left[\frac{1}{2}, -\frac{1}{2}, 1\right].$$

Then $\|\mathbf{w}_2\|^2 = \frac{1}{4} + \frac{1}{4} + 1 = \frac{3}{2}$, and clearing the fraction by factoring out $\frac{1}{2}$,

$$\mathbf{q}_2 = \frac{\frac{1}{2}[1, -1, 2]}{\sqrt{3/2}} = \frac{1}{\sqrt{6}}[1, -1, 2].$$

$k = 3$. Two components to remove:

$$\langle \mathbf{v}_3, \mathbf{q}_1 \rangle = \frac{1}{\sqrt{2}}(0 + 1 + 0) = \frac{1}{\sqrt{2}}, \quad \langle \mathbf{v}_3, \mathbf{q}_2 \rangle = \frac{1}{\sqrt{6}}(0 - 1 + 2) = \frac{1}{\sqrt{6}}.$$

So

$$\mathbf{w}_3 = [0, 1, 1] - \frac{1}{2}[1, 1, 0] - \frac{1}{6}[1, -1, 2] = \left[-\frac{2}{3}, \frac{2}{3}, \frac{2}{3}\right],$$

with $\|\mathbf{w}_3\|^2 = 3 \cdot \frac{4}{9} = \frac{4}{3}$, giving

$$\mathbf{q}_3 = \frac{1}{\sqrt{3}}[-1, 1, 1].$$

Check. All three are unit length: $\frac{1}{2}(1 + 1) = 1$, $\frac{1}{6}(1 + 1 + 4) = 1$, $\frac{1}{3}(1 + 1 + 1) = 1$. And pairwise orthogonal:

$$\langle \mathbf{q}_1, \mathbf{q}_2 \rangle \propto 1 - 1 + 0 = 0, \quad \langle \mathbf{q}_1, \mathbf{q}_3 \rangle \propto -1 + 1 + 0 = 0, \quad \langle \mathbf{q}_2, \mathbf{q}_3 \rangle \propto -1 - 1 + 2 = 0. \quad \checkmark$$

18.4. Reading coordinates with it

The basis was the point. Use it.

Example 18.2. Express $\mathbf{u} = [1, 2, 3]$ in the basis just built. By Theorem 16.1 each coordinate is one dot product:

$$\begin{aligned} \beta_1 &= \langle \mathbf{u}, \mathbf{q}_1 \rangle = \frac{1}{\sqrt{2}}(1 + 2 + 0) = \frac{3}{\sqrt{2}}, \\ \beta_2 &= \langle \mathbf{u}, \mathbf{q}_2 \rangle = \frac{1}{\sqrt{6}}(1 - 2 + 6) = \frac{5}{\sqrt{6}}, \\ \beta_3 &= \langle \mathbf{u}, \mathbf{q}_3 \rangle = \frac{1}{\sqrt{3}}(-1 + 2 + 3) = \frac{4}{\sqrt{3}}. \end{aligned}$$

Reconstructing, with each $\beta_i \mathbf{q}_i$ collapsing to a clean fraction:

$$\frac{3}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}}[1, 1, 0] + \frac{5}{\sqrt{6}} \cdot \frac{1}{\sqrt{6}}[1, -1, 2] + \frac{4}{\sqrt{3}} \cdot \frac{1}{\sqrt{3}}[-1, 1, 1] = \frac{3}{2}[1, 1, 0] + \frac{5}{6}[1, -1, 2] + \frac{4}{3}[-1, 1, 1].$$

Componentwise:

$$\frac{9}{6} + \frac{5}{6} - \frac{8}{6} = 1, \quad \frac{9}{6} - \frac{5}{6} + \frac{8}{6} = 2, \quad 0 + \frac{10}{6} + \frac{8}{6} = 3.$$

So the sum is $[1, 2, 3] = \mathbf{u}$. $\checkmark$

Three dot products, and no system of equations was written down at any point. Compare that with what §16.1 promised: three simultaneous equations in three unknowns. The work moved — it now sits in building the basis, which is §20's subject — but if the basis is to be used more than once, it is paid for only once.

19. What if the vectors are dependent?

Every statement so far has carried the hypothesis that $\mathbf{v}_1, \dots, \mathbf{v}_n$ are linearly independent. It is worth seeing exactly where the algorithm leans on that, because the failure is specific and informative rather than a general collapse.

19.1. The step that fails

Suppose the inputs are dependent, and let $\mathbf{v}_k$ be the first vector in the list that lies in the span of its predecessors:

$$\mathbf{v}_k \in \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_{k-1}\} = \text{span}\{\mathbf{q}_1, \dots, \mathbf{q}_{k-1}\}.$$

The second equality is the span-matching remark of §18.1. So $\mathbf{v}_k$ lies in a subspace for which $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ is already an orthonormal basis — and by Theorem 16.1, its coordinates there are exactly the dot products the algorithm is about to compute:

$$\mathbf{v}_k = \sum_{j=1}^{k-1} \langle \mathbf{v}_k, \mathbf{q}_j \rangle \mathbf{q}_j.$$

That is the entire vector. Line 4 of the pseudocode subtracts it off in full, and line 5 is handed

$$\mathbf{w}_k = \mathbf{v}_k - \mathbf{v}_k = \mathbf{0}.$$

Then $\|\mathbf{w}_k\| = 0$, and the algorithm divides by zero.

The failure is not arbitrary. It is the algorithm reporting, in the only way it can, that $\mathbf{v}_k$ contributed no new direction: everything it had was already spanned, so stripping the known directions left nothing behind. A zero $\mathbf{w}_k$ is the detection of linear dependence.

19.2. In exact arithmetic, and in floating point

Those two settings behave differently, and the difference is the subject of §3–§4.

In **exact arithmetic** $\mathbf{w}_k$ is the zero vector, $\|\mathbf{w}_k\|$ is exactly 0, and the division is undefined. Unpleasant, but unambiguous: a test for $\|\mathbf{w}_k\| = 0$ catches it every time.

In **floating point** it is worse, because $\mathbf{w}_k$ will almost never be exactly zero. The dot products and the subtractions each round (§3.2), so what should have cancelled to nothing instead cancels to a tiny vector of accumulated rounding error — $\|\mathbf{w}_k\|$ might be 10^{-17} rather than 0. The test for exact zero then fails to fire, and line 5 divides by 10^{-17} . The result is a unit vector pointing in a direction determined entirely by rounding noise: it will be dutifully normalised, look perfectly respectable, and mean nothing. Worse, it is then used to strip components off every subsequent vector, so one meaningless direction contaminates the rest of the basis.

A common confusion (A small number is not a zero). This is the pattern from §4: catastrophic cancellation produces a result whose *magnitude* is tiny but whose *relative* error is total. Dividing by such a quantity does not produce a small error — it amplifies pure noise to full size. Testing floating-point quantities for exact equality with zero is almost never the right test.

19.3. What to do about it

Replace the exact test with a tolerance. Between lines 4 and 5, check whether $\|\mathbf{w}\|$ is small *relative* to the vector it came from:

$$\text{if } \|\mathbf{w}\| \leq \varepsilon \|\mathbf{v}_k\|, \quad \text{declare } \mathbf{v}_k \text{ dependent and discard it,}$$

for a small threshold ε tied to the machine precision of §3.3. The comparison is relative rather than absolute because “small” is meaningless on its own — a vector of length 10^{-8} is not small if the input vectors have length 10^{-9} .

With that guard the algorithm no longer fails on dependent input; it does something more useful. It skips the vectors that contribute nothing and returns an orthonormal basis for the span of whatever it was given, however many vectors that takes. If n vectors go in and $r < n$ come out, then r is the dimension of their span, and the $n - r$ discarded vectors were redundant. The procedure has turned into a way of measuring how much independent direction a list of vectors actually contains.

Looking ahead. That count r has a name — the *rank* — and it will matter a great deal when we return to matrices. For now, note only that an algorithm written to build a basis also answers a question nobody asked it: how many of these vectors were doing real work?

20. What it costs

Now count, in the manner of §1 and §4.6. Take the operation to be the FLOP, and let the input be n vectors in $\mathbb{R}^n$.

20.1. One step

Fix k and count what step k does. It runs the inner loop $k - 1$ times, and each pass handles one j and does three things with n -vectors:

Operation	What it costs	FLOPs
$\langle \mathbf{v}_k, \mathbf{q}_j \rangle$	n multiplications, $n - 1$ additions	$2n - 1$
$\langle \mathbf{v}_k, \mathbf{q}_j \rangle \mathbf{q}_j$	scale an n -vector: n multiplications	n
$\mathbf{w} - \text{that}$	subtract two n -vectors: n subtractions	n
per inner pass		$\approx 4n$

So the inner loop of step k costs about $4n(k - 1)$ FLOPs. The normalisation on line 5 then costs one norm (n multiplications, $n - 1$ additions, one square root) plus n divisions — about $3n$ FLOPs, independent of k .

20.2. The total

Sum over $k = 1, \dots, n$. The normalisations contribute $n \cdot 3n = 3n^2$. The inner loops contribute

$$\sum_{k=1}^n 4n(k - 1) = 4n \sum_{k=1}^n (k - 1) = 4n \cdot \frac{n(n - 1)}{2} = 2n^2(n - 1) = 2n^3 - 2n^2.$$

The sum $\sum_{k=1}^n (k - 1) = 0 + 1 + \dots + (n - 1) = n(n - 1)/2$ is the same triangular count that appeared in §2.6. Adding the normalisations,

$$(2n^3 - 2n^2) + 3n^2 = 2n^3 + n^2 = O(n^3).$$

Theorem 20.1 (Cost of Gram–Schmidt). *Orthonormalising n linearly independent vectors in $\mathbb{R}^n$ takes $\approx 2n^3$ FLOPs, so the process runs in $O(n^3)$ time.*

The cubic comes from the shape of the loop, not from any single expensive operation: every step handles n -vectors (n), against every previously fixed direction ($k - 1$, averaging about $n/2$), for every one of the n input vectors. Three nested factors of n .

Remark (Vectors that are not square). Nothing above needed the vectors to live in $\mathbb{R}^n$ specifically. For n vectors in $\mathbb{R}^m$ the same count gives $\approx 2mn^2$ FLOPs: the m tracks the length of each vector, and the n^2 tracks the pairs of directions compared. Setting $m = n$ recovers $2n^3$.

20.3. Where this goes next

Gram–Schmidt is an example of an **orthogonalisation** algorithm: it takes a set of directions that are merely independent and returns a set that is mutually perpendicular, spanning the same space. That is a broad and heavily used category, and this is its first and most transparent member.

Looking ahead. Two threads left open here get picked up later.

The first is the one flagged under the pseudocode. Line 4 dots against $\mathbf{v}_k$; if it dots against the running $\mathbf{w}$ instead, the algorithm is algebraically *identical* — the two differ by multiples of vectors that are orthogonal to $\mathbf{q}_j$ anyway. In floating point they are not identical at all. The version written here is *classical* Gram–Schmidt, and it loses orthogonality as n grows: the later $\mathbf{q}$ ’s drift measurably away from perpendicular. The rearrangement, *modified* Gram–Schmidt, costs the same $2n^3$ FLOPs and holds up far better. Two algorithms that agree exactly in exact arithmetic and disagree in practice is precisely the phenomenon §3–§4 was built to explain, and it is why numerical stability is a subject rather than a footnote. This distinction has real consequences wherever orthogonal bases are computed at scale, including in machine-learning settings.

The second is what the output is *for*. We produced an orthonormal basis and used it to read off coordinates, which was the motivation in §16. But the relationship between the input list and the output list — each $\mathbf{v}_k$ recoverable from $\mathbf{q}_1, \dots, \mathbf{q}_k$, as noted in §18.1 — is itself a factorisation of the matrix whose columns are the $\mathbf{v}_k$. Naming it requires matrix machinery we do not yet have.

We will return to Gram–Schmidt.